

TECNOLÓGICO DE MONTERREY

Maestría en Inteligencia Artificial Aplicada

TC5061 Operaciones de Inteligencia Artificial

Experimentos reproducibles con MLflow para MiBici

Optimización operativa de la red MiBici Guadalajara
2025 y 2026

Proyecto integrador
Segunda entrega

Equipo 30

Ricardo Bahamón
A01840023

Diego Becerril
A01748115

Gregorio Escobedo Straffon
A01840612

Freddin Alcántara
A01797741

25 de octubre de 2026

Repositorio del proyecto

https://github.com/Diegob059/TC5061-Proyecto-Equipo30

1 Resumen de resultados

El proyecto compara modelos para estimar inventario inicial por estación (P1), detectar riesgo de desabasto o saturación (P2) y agrupar estaciones por sus perfiles de uso (P3). El MLflow Project centraliza la ejecución y los registros conservan la relación entre datos, código y modelos seleccionados.

En P1, el árbol con profundidad 10 y hoja mínima 50 reduce el MAE de inventario de prueba de 2.1892 a 1.9861 bicicletas, una mejora de 9.27 %. En P2, LightGBM profundo alcanza macro F1 de 0.3708; su alto recall convive con baja precisión en las alertas críticas. En P3, GMM con dos grupos obtiene silueta de 0.2918 frente a 0.0911 del baseline.

El flujo de trabajo integra comparación de candidatos, selección de modelos, almacenamiento de artefactos y registro mediante el alias champion. P1 utiliza particiones de entrenamiento, validación y prueba; P2 presenta una comparación exploratoria en el periodo denominado test; P3 evalúa la estructura interna de los grupos. Estos protocolos se detallan junto con la interpretación de cada resultado.

Versión de código para reproducción: codex/fix-mlflow-reproducibilidad. El linaje de las corridas documentadas se presenta en la sección 4.

2 MLflow Project y reproducción

MLproject declara los puntos de entrada ingest y train, además de register_p1, register_p2 y register_p3. python_env.yaml establece Python 3.12 y referencia requirements.txt; este último fija, entre otras dependencias, MLflow 3.5.0, scikit-learn 1.7.2 y LightGBM 4.7.0. Las rutas, fechas y semillas se consultan en los módulos de configuración de cada pregunta.

2 1 Preparación del entorno

Ejemplo para una terminal compatible con bash o zsh. El modo local utiliza el entorno activo; no lo crea automáticamente. Se requiere acceso a los datos abiertos de MiBici y, para LightGBM en macOS, una instalación funcional de su dependencia nativa OpenMP.

```
git clone https://github.com/Diegob059/TC5061-Proyecto-Equipo30.git
cd TC5061-Proyecto-Equipo30
git checkout codex/fix-mlflow-reproducibilidad
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

En otra terminal, con el mismo entorno activo, iniciar el servidor. El backend SQLite permite utilizar el Model Registry; la base y los artefactos permanecen fuera del control de versiones.

```
mlflow server --backend-store-uri sqlite:///mlflow.db --port 5001
```

En la terminal de trabajo, apuntar al mismo servidor. Los datos se obtienen mediante ingest y los entrenamientos se ejecutan mediante el Project (MLflow, s. f.-a).

```
export MLFLOW_TRACKING_URI=http://127.0.0.1:5001
mlflow run . -e ingest -P question=all --env-manager local \
  --experiment-name mibici-equipo30

mlflow run . -e train -P question=p1 --env-manager local \
  --experiment-name mibici-equipo30
mlflow run . -e train -P question=p2 --env-manager local \
  --experiment-name mibici-equipo30
mlflow run . -e train -P question=p3 --env-manager local \
  --experiment-name mibici-equipo30
```

2 2 Verificación funcional

La ejecución de P1 mediante MLflow Project completó la búsqueda de candidatos y seleccionó el árbol d10 h50. La comparación de 118 métricas contra las evidencias de referencia produjo una diferencia máxima de 0.0. El registro, la carga por alias y la predicción se verificaron en el servidor local. La suite del proyecto aprobó 36 pruebas, incluida una prueba de entrenamiento, guardado, carga y promoción del modelo.

La comprobación se realizó con el entorno fijado y los datos procesados disponibles. En macOS, LightGBM requiere OpenMP; su instalación se realiza con el comando brew install libomp. Los resultados de P2 y P3 se documentan a partir de sus corridas registradas.

Los datos de MiBici, mlflow.db, mlruns/, mlartifacts/ y los modelos serializados se mantienen fuera del entregable y del control de versiones.

3 Comparación y selección de modelos

3 1 P1 Inventario inicial por estación

Se utilizan 7,893,339 viajes de enero de 2025 a septiembre de 2026. La cohorte fija comprende 368 estaciones con salidas en enero de 2025. El entrenamiento cubre febrero a diciembre de 2025 (1,474,944 filas); la validación, enero a junio de 2026 (799,296); y la prueba, julio a septiembre de 2026 (406,272). Las franjas pico se determinan con 2025: 7 a 10 h en días laborables y 9 a 12 h en fines de semana.

El modelo predice salidas y llegadas por hora. Un balance posterior transforma esos flujos en inventario inicial requerido. La selección utiliza exclusivamente el menor MAE de inventario en validación; el margen de seguridad se calibra con el cuantil 0.90 de subestimación en esa misma partición. Se comparan un baseline semanal y cinco alternativas, cuatro de ellas en una búsqueda con corridas anidadas.

Tabla 1. Comparación de inventario. MAE y RMSE en bicicletas; Δ es la reducción relativa del MAE de prueba frente al baseline. d indica profundidad y h, mínimo de muestras por hoja.

Modelo	MAE validación	MAE prueba	RMSE prueba	Δ prueba
Semanal baseline	2.1726	2.1892	3.3306	—
Histórico	1.9430	2.0481	3.0851	6.44 %
Árbol d6 h50	1.9722	2.0441	3.1599	6.63 %
Árbol d6 h100	1.9714	2.0426	3.1554	6.69 %
Árbol d10 h50	1.9263	1.9861	3.0548	9.27 %
Árbol d10 h100	1.9293	1.9879	3.0724	9.19 %

Se selecciona el árbol d10 h50: MAE de validación de 1.9263, una reducción de 11.34 % frente a 2.1726 del baseline. En prueba obtiene 1.9861 y mejora 9.27 %. El resultado conecta la selección con el error de inventario que afecta la decisión de abastecimiento, en lugar de elegir únicamente por el ajuste de los flujos horarios.

Tabla 2. Estabilidad y costo de P1. Los tiempos son mediciones locales y no representan latencia de un servicio desplegado.

Medida	Baseline	Árbol d10 h50
MAE del peor mes de validación	2.2682	2.0093
Desviación mensual del MAE en validación	0.0792	0.0642
Ajuste del modelo en segundos	0.0190	1.9314
Predicción en ms por 1,000 filas	0.0147	0.0675

El árbol d6 h100 presenta menor dispersión mensual (0.0581), pero mayor MAE. Se favorece d10 h50 por su menor error y menor peor mes, aceptando ese intercambio. La cobertura de conteos de prueba es 92.77 % y no limita por capacidad física. El modelo no observa demanda rechazada ni rebalanceos del operador; aplica un orden conservador de salidas antes de llegadas dentro de cada hora. La capacidad del catálogo de septiembre es una aproximación. Estas condiciones impiden interpretar la cobertura como garantía de disponibilidad.

Fuente: reports/p1/avanzado/comparacion.csv, seleccion.json, registry.json y verificacion.json. El baseline avanzado se reevalúa con el mismo protocolo de inventario; no se mezclan las métricas anteriores de flujos con las de esta tabla.

3 2 P2 Riesgo de desabasto y saturación

Se comparan un baseline estratificado y tres configuraciones de LightGBM (Ke et al., 2017), con 4,037,300 filas de entrenamiento y 3,078,500 de prueba. El entrenamiento termina antes de 2025-12-31 22:00:00 y la prueba comienza el 1 de enero de 2026. Ese corte evita que las etiquetas con horizonte de dos horas invadan la partición siguiente.

Tabla 3. Métricas de P2 en la partición denominada test. Δ F1 es diferencia absoluta frente al baseline. No se registró una partición independiente de validación.

Modelo	Macro F1	Balanced accuracy	Recall desabasto	Recall saturación	Δ F1
Baseline estratificado	0.3333	0.3336	0.78 %	0.62 %	—
LightGBM balanceado	0.3662	0.8746	87.90 %	90.58 %	+0.0329
LightGBM profundo	0.3708	0.8800	88.41 %	90.92 %	+0.0375
LightGBM compacto	0.3578	0.8588	85.64 %	89.51 %	+0.0245

Tabla 4. Hiperparámetros y tiempos de P2. η es learning_rate; las latencias corresponden a predicción por lotes, sin red ni aplicación.

Candidato	Árboles	Prof.	Hojas	η	Ajuste s	ms por 1,000
balanceado	120	7	31	0.08	11.74	1.7616
profundo	180	9	63	0.04	22.95	3.9791
compacto	80	5	16	0.1	7.04	0.8719

La configuración profunda obtiene el mayor macro F1 (0.3708), mejora absoluta de 0.0375 y balanced accuracy de 0.8800. Su ajuste tarda 22.95 s y la predicción 3.9791 ms por 1,000 filas. La alternativa compacta cuesta menos, pero reduce el macro F1 a 0.3578. No hay repeticiones por semilla ni intervalos para afirmar estabilidad estadística.

Tabla 5. Calidad de las alertas del modelo profundo, calculada sobre la misma partición test.

Clase	Soporte real	Precisión	Recall
Normal	3,049,794	99.94 %	84.66 %
Desabasto	11,897	4.44 %	88.41 %
Saturación	16,809	5.92 %	90.92 %

La sensibilidad aumenta frente al baseline, pero 95.56 % de las predicciones de desabasto y 94.08 % de las de saturación son falsas alarmas. Una política que enviara un camión por cada alerta tendría una carga operativa elevada. Conviene calibrar umbrales y priorizar alertas según el costo de intervención antes de usar el resultado para asignar recursos.

El código elige al ganador usando las métricas de test. Esa reutilización introduce sesgo de selección, aun con la frontera temporal corregida. Las cifras son comparaciones exploratorias y requieren evaluación posterior en un periodo reservado. Se conserva el champion registrado con esta limitación.

Fuente: reports/p2/comparacion.csv y seleccion.json; reports/p2_classification_report_lgbm_profundo.txt. Los valores de validación se consideran no disponibles, en lugar de asignarles métricas de prueba.

3 3 P3 Agrupamiento de estaciones

Se analizan 352 estaciones mediante perfiles agregados y estandarización con StandardScaler. La búsqueda anidada comprende 36 combinaciones: KMeans y mezclas gaussianas (GMM), valores de k de 2 a 10 y dos conjuntos de características. hourly contiene 52 variables; bucketed, 14. De esa búsqueda se conservan tres candidatas para comparar contra KMeans horario con k igual a 4.

La silueta resume cohesión y separación; un valor mayor favorece la agrupación. El índice Davies–Bouldin se minimiza. Ambos se calculan sobre los mismos perfiles utilizados para ajustar los grupos, por lo que miden calidad interna y no desempeño sobre meses o estaciones reservados (scikit-learn developers, s. f.).

Tabla 6. Evaluación interna de P3. Δ silueta expresa diferencia absoluta contra el baseline. Las columnas de validación y prueba temporal no están disponibles en este protocolo.

Modelo	Variables	k	Silueta	Davies–Bouldin	Δ silueta
KMeans horario baseline	52	4	0.0911	2.3217	—
GMM horario	52	2	0.2918	1.9792	+0.2006
KMeans por franjas	14	2	0.1573	2.1235	+0.0662
GMM por franjas	14	2	0.1536	2.4225	+0.0624

Se selecciona GMM horario con k igual a 2: silueta de 0.2918 y Davies–Bouldin de 1.9792. Frente al baseline, la silueta aumenta 0.2006 puntos (220.13 % relativo) y Davies–Bouldin disminuye aproximadamente 14.75 %. El porcentaje relativo es grande porque la referencia es baja; una silueta de 0.2918 todavía indica separación limitada. La comparación más directa es entre los dos modelos horarios: los conjuntos de 14 y 52 variables definen geometrías distintas.

El informe de clustering registra 329 estaciones en el grupo 0 y 23 en el grupo 1 (93.47 % y 6.53 %). Ese desequilibrio justifica examinar estaciones atípicas y la influencia de las variables de volumen antes de definir estrategias operativas distintas. La hipótesis de zonas residenciales o laborales requiere contrastar horarios, ubicación y uso de suelo; no se considera una etiqueta observada.

Los grupos pueden orientar el análisis de rebalanceo, pero no se midieron ahorros de distancia, tiempo ni costo de camiones. Tampoco se registraron tiempos comparables de ajuste e inferencia ni estabilidad entre semillas o periodos. La selección se sostiene en las métricas internas disponibles; para evaluar generalización se requiere ajustar perfiles históricos y comprobar persistencia y utilidad en un periodo posterior.

3 4 Artefactos para interpretar los resultados

P1 conserva configuración, manifiesto, fuente y gráficas de flujos e inventario, además de firma, ejemplo de entrada y dataset registrado. P2 aporta informes por clase, matrices de confusión y configuraciones por candidato. P3 aporta selección de k, perfiles y mapa de estaciones. Estos archivos permiten revisar el cálculo; su presencia no sustituye la inspección del modelo y de su firma en el servidor correspondiente.

Fuentes: reports/p3/comparacion.csv, seleccion.json, registry.json y reports/p3_clustering_report.txt. La inercia de KMeans no se compara con GMM porque los objetivos de ajuste son distintos.

4 Model Registry y ciclo de vida

El equipo utiliza el alias champion para identificar la versión seleccionada de cada pregunta. El alias permite cambiar la referencia de consumo sin modificar el nombre del modelo en el cliente; la versión numérica conserva la identidad del artefacto. Este esquema sigue el flujo actual de aliases y etiquetas de MLflow (MLflow, s. f.-b). Su asignación representa la designación de producción dentro del ejercicio; no acredita por sí sola un servicio desplegado ni una validación operativa.

Tabla 7. Estado declarado en los archivos registry.json del repositorio. Los números de versión pertenecen a los servidores en los que se registraron los modelos y pueden variar al recrear el entorno.

Nombre registrado	Versión	Alias	Carga y predicción
mibici-equipo30-p1-demanda	1	champion	5 filas
mibici-equipo30-p2-riesgo	3	champion	5 filas
mibici-equipo30-p3-grupos	3	champion	5 filas

Cada archivo de registro conserva experimento, corrida de origen, commit y versión de datos. Las tres referencias pertenecen al experimento mibici-equipo30, identificado como 1 en sus evidencias locales. El identificador numérico del experimento no garantiza que sea una única base de tracking compartida.

4 1 Linaje de P1

Árbol d10 h50. Referencia de consumo:

```
models:/mibici-equipo30-p1-demanda@champion
```

```
Corrida  e17f484bf7b349d19a63edc19802cb07
Commit   667444a010847d64067a9596986de52115a8b112
Datos    p1-b82961c841c7089205efb9ef8de938995258201f2ccf81344756af16bdb1241c
```

4 2 Linaje de P2

LightGBM profundo. Referencia de consumo:

```
models:/mibici-equipo30-p2-riesgo@champion
```

```
Corrida  8350c8c6b05d49f9a2f1b7746bd0463c
Commit   5bae625746700ae29c8ab53230b816f4d56a5c09
Datos    p2-mibici-v2025_2026-ddfa3aa49c
```

4 3 Linaje de P3

GMM horario con dos grupos. Referencia de consumo:

```
models:/mibici-equipo30-p3-grupos@champion
```

```
Corrida  f732c016be2d4e80b05415d005826f4e
Commit   f967a70a5b094e5deee55eb7dbff7c26c72c6a14
Datos    mibici-v2025_2026-f1db9c0782
```

La verificación registrada consiste en cargar por alias y ejecutar predict sobre cinco ejemplos. En P2 se documentan las clases 0, 1 y 2. En P3 los cinco ejemplos devuelven el grupo 0; esa muestra no prueba la cobertura de ambos grupos ni el comportamiento en las 352 estaciones. La comprobación de carga es una prueba funcional mínima y debe complementarse con evaluación de calidad y monitoreo.

Fuentes: reports/p1/avanzado/registry.json, reports/p2/registry.json y reports/p3/registry.json. Los identificadores se transcriben íntegros para evitar confundir la corrida seleccionada con otra versión del mismo candidato.

4 4 Ejecución de las configuraciones elegidas

Con los datos preparados, el servidor disponible y el entorno activo, estos comandos fijan los hiperparámetros principales de los modelos seleccionados. Se utiliza la versión de código indicada en la sección 2.

```
mlflow run . -e train -P question=p1 -P model=arbol \
  -P max_depth=10 -P min_samples_leaf=50 \
  --env-manager local --experiment-name mibici-equipo30

mlflow run . -e train -P question=p2 -P model=lgbm \
  -P n_estimators=180 -P max_depth=9 \
  -P num_leaves=63 -P learning_rate=0.04 \
  --env-manager local --experiment-name mibici-equipo30

mlflow run . -e train -P question=p3 -P model=gmm \
  -P k=2 -P feature_set=hourly \
  --env-manager local --experiment-name mibici-equipo30
```

Para reconstruir las tablas de comparación se deben ejecutar las búsquedas completas de la sección 2. Los run_id del reporte identifican corridas históricas y no son reutilizables en una base nueva. Después de completar cada búsqueda, seleccion.json contiene la corrida elegida del entorno recién ejecutado. El siguiente bloque registra esas corridas mediante el Project; no debe utilizarse si el entrenamiento falló o si los JSON corresponden a otro servidor.

```
python - <<'PY'
import json
import subprocess
from pathlib import Path

for q, folder in [('p1', 'p1/avanzado'), ('p2', 'p2'),
                  ('p3', 'p3')]:
    path = Path('reports') / folder / 'seleccion.json'
    run_id = json.loads(path.read_text())['selected_run_id']
    subprocess.run([
        'mlflow', 'run', '.', '-e', f'register_{q}',
        '-P', f'run_id={run_id}',
        '-P', f'reason=Modelo seleccionado de {q}',
        '--env-manager', 'local',
        '--experiment-name', 'mibici-equipo30'
    ], check=True)
PY
```

La consulta por alias permite cargar las referencias de consumo; cada modelo requiere datos con su propia firma, no una misma matriz para las tres preguntas.

```
import mlflow.pyfunc
for name in ['mibici-equipo30-p1-demanda',
             'mibici-equipo30-p2-riesgo',
             'mibici-equipo30-p3-grupos']:
    model = mlflow.pyfunc.load_model(f'models:/{name}@champion')
```

El registro requiere que los artefactos del run sean accesibles desde el servidor. Los archivos JSON de evidencia documentan las ejecuciones previas, pero no contienen los modelos serializados ni reconstruyen una base de tracking por sí solos.

5 Evidencia de MLflow

5 1 Corridas anidadas y artefactos

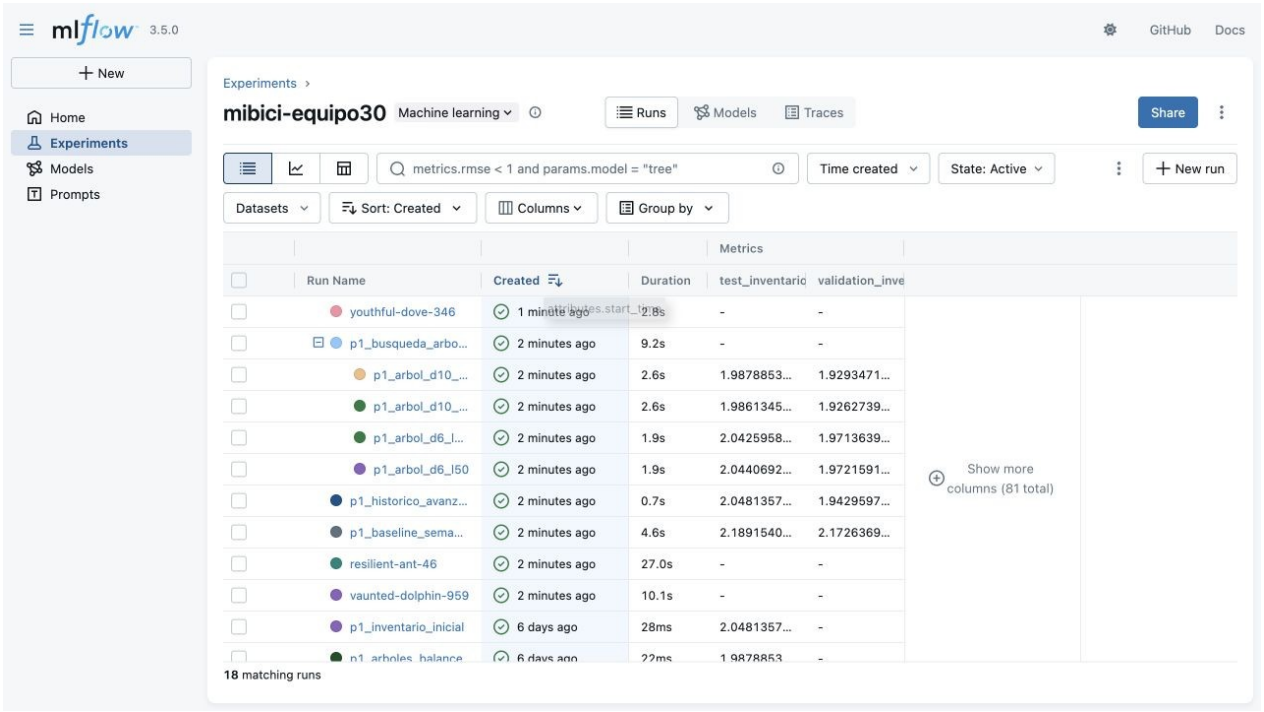


Figura 1. Experimento mibici-equipo30 en MLflow 3.5.0. La búsqueda de árboles de P1 aparece expandida con cuatro corridas hijas; las columnas visibles muestran métricas de validación y prueba. Los valores completos se presentan en la tabla 1.

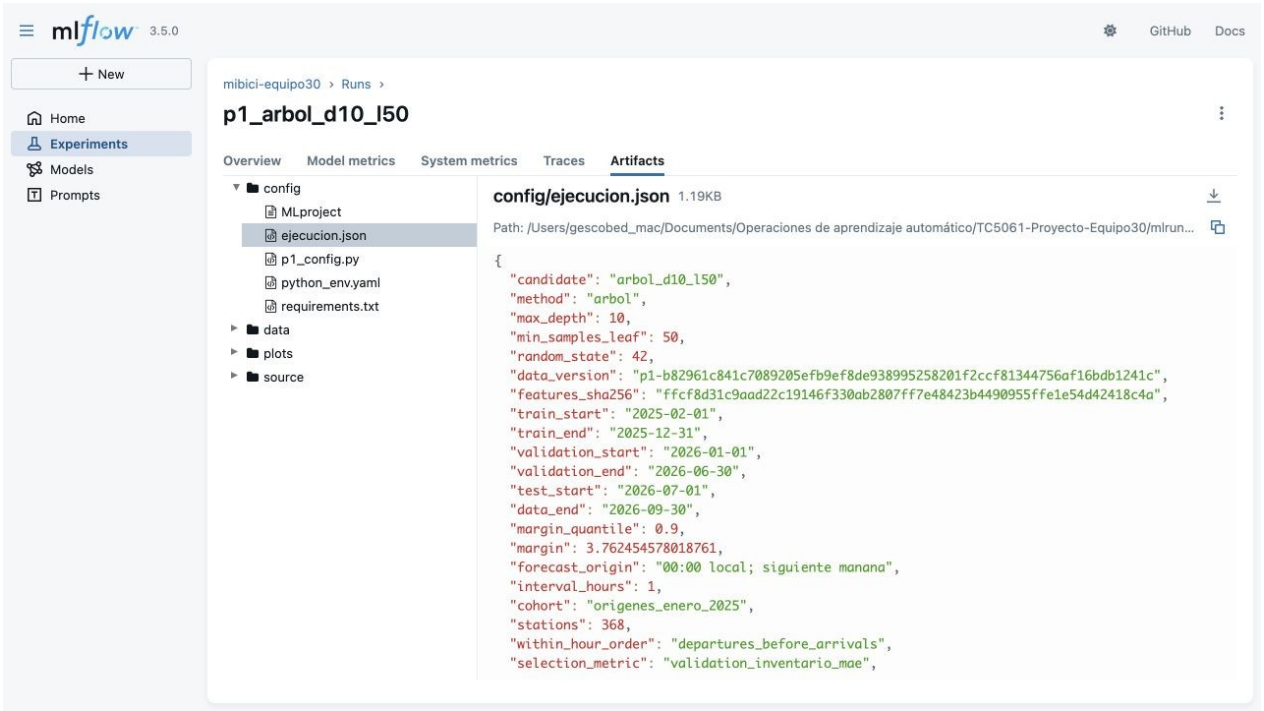


Figura 2. Pestaña Artifacts del candidato p1_arbol_d10_l50. config/ejecucion.json muestra hiperparámetros, versión de datos, hash de características y fechas de partición. El árbol lateral permite identificar las carpetas config, data, plots y source.

Fuente de ambas capturas: reports/figures/p1/avanzado/. Las capturas corresponden a las corridas de referencia de P1 y se relacionan con las métricas de la tabla 1.

5 2 Registro y trazabilidad de corridas

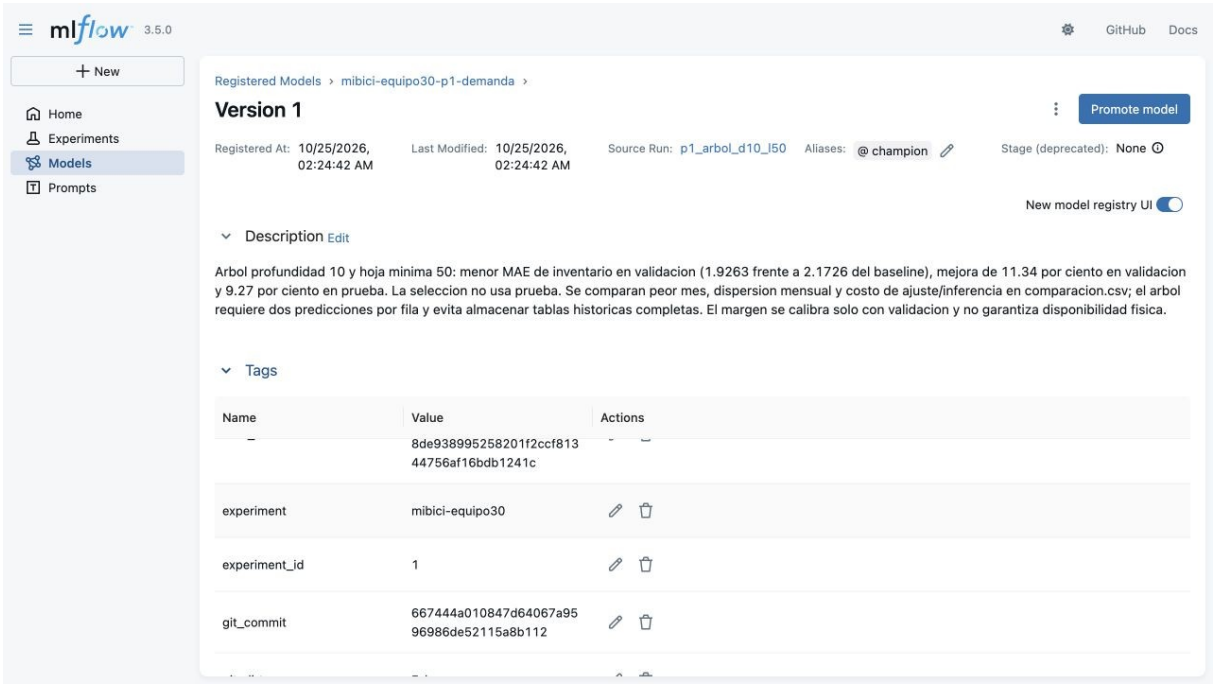


Figura 3. Versión 1 de mibici-equipo30-p1-demanda. Se observan el alias champion, la corrida de origen y la etiqueta git_commit del modelo seleccionado. La versión de datos completa se transcribe en la sección 4.

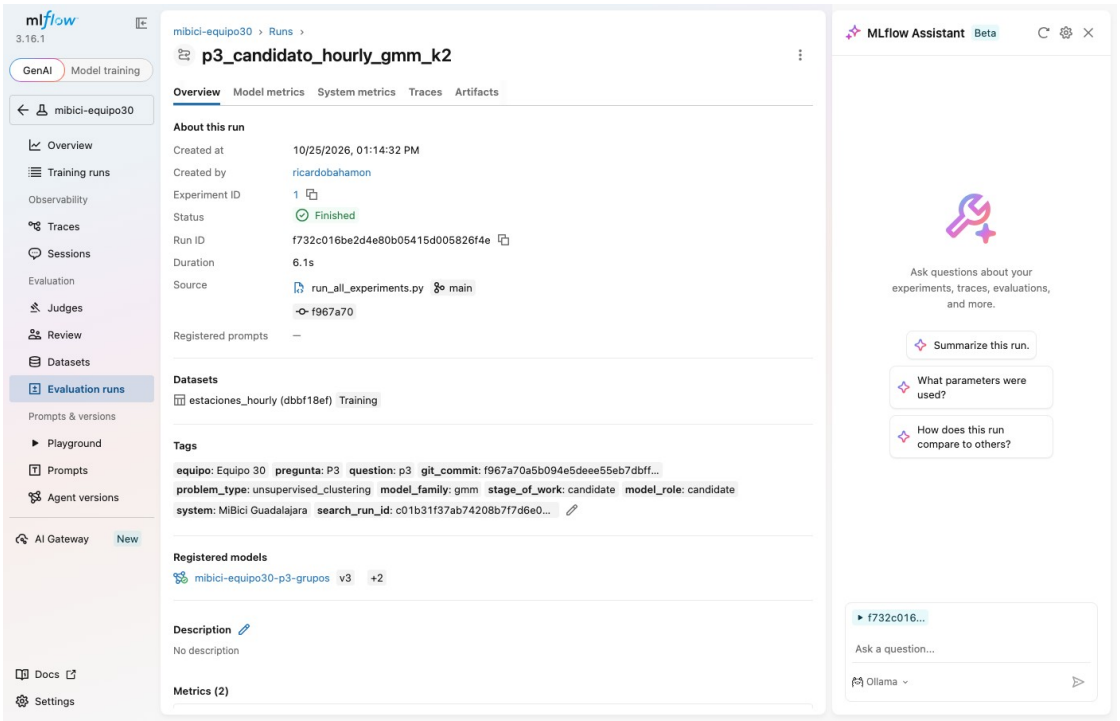


Figura 4. Detalle de la corrida candidata de P3 en MLflow 3.16.1. La vista muestra la corrida f732c016..., el commit f967a70 y el vínculo a la versión 3 del modelo registrado.

La tabla 7 sintetiza los tres modelos y sus aliases. Los archivos registry.json conservan las referencias completas de versión, corrida, commit y datos que se transcriben en la sección 4.

6 Conclusiones

El MLflow Project organiza la ingesta, el entrenamiento y el registro de modelos para las tres preguntas del proyecto. La separación entre configuración, datos, código y artefactos permite relacionar los resultados con las condiciones de cada ejecución. El Model Registry conserva el linaje y el alias champion establece una referencia de consumo para cada modelo seleccionado.

Para P1, el árbol d10 h50 reduce el MAE de inventario en prueba de 2.1892 a 1.9861 bicicletas, equivalente a una mejora de 9.27 % frente al baseline. Su elección utiliza validación temporal y se respalda con la evaluación mensual de error y el costo de inferencia. La ejecución funcional verificada completa entrenamiento, registro y carga por alias.

Para P2, LightGBM profundo obtiene el mayor macro F1 de las configuraciones comparadas, con 0.3708, y mejora la sensibilidad ante desabasto y saturación. La baja precisión de las alertas exige calibrar su uso según el costo de intervención. Las métricas del periodo usado para comparar candidatos se interpretan como evidencia exploratoria; una evaluación futura con un periodo reservado permitirá medir generalización.

Para P3, GMM horario con dos grupos mejora la silueta de 0.0911 a 0.2918 y reduce Davies–Bouldin respecto al baseline. Esta estructura apoya la exploración de perfiles de estaciones. Su utilidad para el rebalanceo se evaluará contrastando la persistencia de los grupos entre periodos y su relación con decisiones operativas.

Los resultados muestran mejoras medibles frente a las referencias de cada pregunta. La documentación de hiperparámetros, métricas, artefactos y linaje facilita revisar las decisiones y continuar el proyecto desde versiones identificables. Las siguientes etapas pueden utilizar las referencias champion para integrar predicciones y evaluar su impacto en la operación de MiBici.

7 Referencias

Equipo 30. (2026). *TC5061 Proyecto Equipo 30* [Código fuente y evidencias de experimentación; commit ab88e0a]. GitHub.

https://github.com/Diegob059/TC5061-Proyecto-Equipo30/tree/ab88e0a54d615b2892117bba32299a7ad0ccf2ea

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. En *Advances in Neural Information Processing Systems* (Vol. 30).

https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

MiBici. (s. f.). *Datos abiertos* [Conjunto de datos]. Recuperado el 25 de octubre de 2026, de

https://www.mibici.net/es/datos-abiertos/

MLflow. (s. f.-a). *ML Projects*. Recuperado el 25 de octubre de 2026, de

https://mlflow.org/docs/latest/ml/projects/

MLflow. (s. f.-b). *Model Registry workflows*. Recuperado el 25 de octubre de 2026, de

https://www.mlflow.org/docs/latest/ml/model-registry/workflow/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

https://www.jmlr.org/papers/v12/pedregosa11a.html

scikit-learn developers. (s. f.). *Clustering performance evaluation*. Recuperado el 25 de octubre de 2026, de

https://scikit-learn.org/stable/modules/clustering.html#clustering-performance-evaluation

Fuentes de resultados y criterio de transcripción

Las cifras proceden de los CSV y JSON del repositorio fijado, no de valores estimados a partir de capturas. Los porcentajes de mejora se calculan contra el baseline de la misma tabla. Los tiempos son los registrados por cada ejecución y no se comparan como un benchmark entre preguntas ni entre equipos. La implementación utiliza herramientas de scikit-learn (Pedregosa et al., 2011).

Reparación verificada: commits 5d377c1 y d58fa75; reports/verificacion_reparacion_mlflow.json conserva los identificadores de la ejecución nueva. La tabla de linaje de la sección 4 mantiene las corridas históricas, sin sustituirlas por las de la verificación. Estas reparaciones no cambian sus métricas.